



GUÍA COMPLETA: REPLICAR MCP SERVER SUPERIOR PARA IRIS



ÍNDICE

1. [Visión General](#)
 2. [Componentes a Replicar](#)
 3. [Implementación Paso a Paso](#)
 4. [Integración con IRIS](#)
 5. [Configuración de APIs](#)
 6. [Testing y Validación](#)
 7. [Despliegue](#)
-



VISIÓN GENERAL

Objetivo:

Integrar el MCP Server Superior en la plataforma IRIS para mejorar:

-  **Visibilidad en tiempo real** con dashboard avanzado
-  **Automatización inteligente** con templates
-  **Gestión centralizada** con CLI avanzada
-  **Notificaciones inteligentes** multi-canal
-  **Setup wizard** para configuración rápida

Componentes Clave:

1. **Dashboard React** con métricas en tiempo real
 2. **Sistema de Templates** para automatización
 3. **CLI Avanzada** para gestión
 4. **Sistema de Notificaciones** multi-canal
 5. **Setup Wizard** para instalación automática
-

COMPONENTES A REPLICAR

1. DASHBOARD REACT AVANZADO

Archivos base del MCP Server:

- `mcp-dashboard/src/components/Dashboard.tsx` (281 líneas)
- `mcp-dashboard/src/components/Charts.tsx`
- `mcp-dashboard/src/server/metrics_server.py` (FastAPI + SSE)

Funcionalidades a implementar:

- Métricas en tiempo real de agentes IRIS
- Monitoreo de tokens procesados
- Estados de tareas multiagente
- Performance dashboard por agente (Ventas, Soporte, Consultoría)
- Alertas y notificaciones visuales

2. SISTEMA DE TEMPLATES

Archivo base:

- `templates.py` (25,361 bytes, 17 funciones)

Templates para IRIS:

- Template de Automatización de Ventas
- Template de Procesamiento de Soporte
- Template de Análisis de Consultoría
- Template de Configuración Multiagente
- Template de Optimización de Flujos

3. CLI AVANZADA

Archivo base:

- `cli.py` (19,296 bytes, 20+ funciones)

Comandos específicos para IRIS:

```
iris status                # Estado de agentes
iris deploy <agente>       # Desplegar agente específico
iris metrics               # Métricas en tiempo real
iris template <nombre>    # Crear desde template
iris notify <tipo>        # Configurar notificaciones
iris log <agente>         # Ver logs de agente
```

4. SISTEMA DE NOTIFICACIONES

Archivo base:

- `notifications.py` (20,392 bytes, 28 funciones)

Integración con IRIS:

- Notificaciones por email sobre estado de agentes
- Webhooks para integraciones externas
- Alertas en consola durante automatización
- Monitoreo automático de performance

5. SETUP WIZARD

Archivo base:

- `setup_wizard.py` (17,114 bytes, 17 funciones)

Wizard específico para IRIS:

- Detección automática de configuración IRIS
 - Configuración de APIs de MiniMax
 - Setup de agentes especializados
 - Instalación de dependencias
-



IMPLEMENTACIÓN PASO A PASO

PASO 1: Preparar Entorno de Desarrollo

```
# 1. Crear estructura de directorios
mkdir iris-mcp-integration/
cd iris-mcp-integration/

# 2. Configurar Node.js para dashboard
npm init -y
npm install react react-dom typescript @types/react @types/react-dom
npm install recharts axios
npm install -D @types/node vite @vitejs/plugin-react

# 3. Configurar Python para backend
python -m venv iris-mcp-env
source iris-mcp-env/bin/activate # Linux/Mac
# iris-mcp-env\Scripts\activate # Windows
pip install fastapi uvicorn requests python-dotenv
```

PASO 2: Implementar Dashboard React

2.1 Crear estructura de componentes:

```
mkdir -p src/components src/hooks src/utils src/types
```

2.2 Componente Principal Dashboard.tsx:

```

// Base del MCP Server adaptado para IRIS
import { useState, useEffect } from 'react';
import { LineChart, Line, XAxis, YAxis, CartesianGrid, Tooltip,
ResponsiveContainer } from 'recharts';
import { RefreshCw, Activity, Users, TrendingUp, AlertCircle } from
' lucide-react';

interface AgentMetrics {
  agent: string;
  status: 'active' | 'idle' | 'error';
  tasksCompleted: number;
  avgResponseTime: number;
  tokenUsage: number;
  lastActivity: string;
}

interface DashboardProps {
  apiBase: string;
}

const IrisDashboard: React.FC<DashboardProps> = ({ apiBase }) => {
  const [agents, setAgents] = useState<AgentMetrics[]>([]);
  const [loading, setLoading] = useState(true);
  const [realTimeData, setRealTimeData] = useState<any[]>([]);

  useEffect(() => {
    // Conectar a SSE para datos en tiempo real
    const eventSource = new EventSource(`${apiBase}/metrics/stream`);

    eventSource.onmessage = (event) => {
      const data = JSON.parse(event.data);
      setAgents(data.agents);
      setRealTimeData(prev => [...prev.slice(-19), data.timestamp]);
    };

    return () => eventSource.close();
  }, [apiBase]);

  // Resto del componente...
};

```

2.3 Configurar servidor de métricas:

```

# metrics_server.py adaptado para IRIS
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
import uvicorn
import json
from datetime import datetime
import asyncio

app = FastAPI(title="IRIS MCP Metrics Server")

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

@app.get("/metrics/stream")
async def metrics_stream():
    async def generate():
        while True:
            # Generar métricas específicas de IRIS
            metrics = {
                "timestamp": datetime.now().isoformat(),
                "agents": [
                    {
                        "agent": "Sales Agent",
                        "status": "active",
                        "tasksCompleted": 156,
                        "avgResponseTime": 1.2,
                        "tokenUsage": 45200
                    },
                    {
                        "agent": "Support Agent",
                        "status": "active",
                        "tasksCompleted": 89,
                        "avgResponseTime": 0.8,
                        "tokenUsage": 32400
                    },
                    {
                        "agent": "Consulting Agent",

```

```

        "status": "active",
        "tasksCompleted": 23,
        "avgResponseTime": 2.1,
        "tokenUsage": 67800
    }
]
}
yield f"data: {json.dumps(metrics)}\n\n"
await asyncio.sleep(2)

return StreamingResponse(generate(), media_type="text/plain")

if __name__ == "__main__":
    uvicorn.run(app, host="0.0.0.0", port=8000)

```

PASO 3: Implementar Sistema de Templates

3.1 Crear TemplateManager adaptado:


```

# iris_templates.py
import json
from typing import Dict, Any
from pathlib import Path

class IRISTemplateManager:
    def __init__(self):
        self.templates_dir = Path("iris_templates")
        self.templates_dir.mkdir(exist_ok=True)

    def create_sales_automation_template(self) -> Dict[str, Any]:
        return {
            "name": "Sales Automation Template",
            "description": "Automatización completa de procesos de
venta",
            "agents": {
                "lead_qualification": {
                    "model": "minimax/m1",
                    "prompts": ["Qualify leads based on criteria"],
                    "actions": ["score_lead", "route_to_sales",
"schedule_followup"]
                },
                "proposal_generator": {
                    "model": "minimax/m2",
                    "prompts": ["Generate sales proposals"],
                    "actions": ["create_proposal", "customize_pricing",
"send_to_prospect"]
                },
                "follow_up_automation": {
                    "model": "minimax/m1",
                    "prompts": ["Manage follow-up communications"],
                    "actions": ["send_followups", "update_crm",
"schedule_meetings"]
                }
            },
            "workflow": [
                "lead_qualification",
                "proposal_generator",
                "follow_up_automation"
            ]
        }

```

```

def create_support_template(self) -> Dict[str, Any]:
    return {
        "name": "Support Automation Template",
        "description": "Automatización de atención al cliente",
        "agents": {
            "ticket_classifier": {
                "model": "minimax/m1",
                "prompts": ["Classify support tickets"],
                "actions": ["categorize_ticket", "assign_priority"],
"route_to_agent"]
            },
            "response_generator": {
                "model": "minimax/m2",
                "prompts": ["Generate helpful responses"],
                "actions": ["draft_response", "suggest_solutions",
"escalate_if_needed"]
            }
        }
    }

def create_consulting_template(self) -> Dict[str, Any]:
    return {
        "name": "Consulting Analysis Template",
        "description": "Análisis avanzado para consultoría",
        "agents": {
            "data_analyzer": {
                "model": "minimax/m2",
                "prompts": ["Analyze business data"],
                "actions": ["process_data", "identify_patterns",
"generate_insights"]
            },
            "report_generator": {
                "model": "minimax/m2",
                "prompts": ["Create professional reports"],
                "actions": ["compile_findings",
"create_visualizations", "generate_summary"]
            }
        }
    }

def save_template(self, template: Dict[str, Any]) -> str:
    template_id = template["name"].lower().replace(" ", "_")

```

```
file_path = self.templates_dir / f"{template_id}.json"

with open(file_path, 'w') as f:
    json.dump(template, f, indent=2)

return str(file_path)
```

PASO 4: Implementar CLI Avanzada

4.1 Crear CLI principal:

```

# iris_cli.py
import click
import asyncio
import json
from pathlib import Path
from iris_templates import IRISTemplateManager

@click.group()
@click.option('--api-base', default='http://localhost:8000', help='API
base URL')
@click.pass_context
def iris_cli(ctx, api_base):
    ctx.ensure_object(dict)
    ctx.obj['api_base'] = api_base

@iris_cli.command()
@click.pass_context
def status(ctx):
    """Mostrar estado de todos los agentes IRIS"""
    api_base = ctx.obj['api_base']
    click.echo("🔍 Consultando estado de agentes IRIS...")

    # Aquí iría la lógica para consultar la API de IRIS
    agents = [
        {"name": "Sales Agent", "status": "active", "tasks": 156},
        {"name": "Support Agent", "status": "active", "tasks": 89},
        {"name": "Consulting Agent", "status": "idle", "tasks": 23}
    ]

    for agent in agents:
        status_icon = "🟢" if agent['status'] == 'active' else "🔴"
        click.echo(f"{status_icon} {agent['name']}: {agent['status']}
({agent['tasks']} tareas)")

@iris_cli.command()
@click.argument('agent_name')
@click.pass_context
def deploy(ctx, agent_name):
    """Desplegar agente específico"""
    click.echo(f"🚀 Desplegando agente: {agent_name}")

# Lógica de despliegue

```

```

    click.echo(f"✅ Agente {agent_name} desplegado exitosamente")

@iris_cli.command()
@click.pass_context
def metrics(ctx):
    """Mostrar métricas en tiempo real"""
    api_base = ctx.obj['api_base']
    click.echo("📊 Métricas en tiempo real de IRIS")

    # Aquí iría la conexión con la API de métricas
    click.echo("🟢 Sistema: Operativo")
    click.echo("⚡ Tokens procesados hoy: 145,320")
    click.echo("🤖 Agentes activos: 3/3")
    click.echo("📈 Tareas completadas: 1,247")

@iris_cli.command()
@click.argument('template_name')
@click.pass_context
def template(ctx, template_name):
    """Crear configuración desde template"""
    click.echo(f"📄 Creando template: {template_name}")

    manager = IRISTemplateManager()

    if template_name == "sales":
        template = manager.create_sales_automation_template()
    elif template_name == "support":
        template = manager.create_support_template()
    elif template_name == "consulting":
        template = manager.create_consulting_template()
    else:
        click.echo(f"❌ Template '{template_name}' no encontrado")
        return

    manager.save_template(template)
    click.echo(f"✅ Template {template_name} creado exitosamente")

@iris_cli.command()
@click.option('--type', 'notification_type',
              type=click.Choice(['email', 'webhook', 'console']),
              default='console', help='Tipo de notificación')
@click.pass_context

```

```

def notify(ctx, notification_type):
    """Configurar sistema de notificaciones"""
    click.echo(f"🔔 Configurando notificaciones: {notification_type}")

    # Lógica de configuración de notificaciones
    if notification_type == 'email':
        click.echo("✉️ Configura email: [ir_toggle_smtp]")
    elif notification_type == 'webhook':
        click.echo("🔗 Configura webhook: [ir_set_webhook_url]")

@iris_cli.command()
@click.argument('agent_name', required=False)
@click.pass_context
def log(ctx, agent_name):
    """Ver logs de agente específico o todos"""
    if agent_name:
        click.echo(f"📋 Logs de {agent_name}")
    else:
        click.echo("📋 Logs de todos los agentes")

    # Lógica para obtener logs
    click.echo("🔍 Mostrando últimos logs...")

if __name__ == '__main__':
    iris_cli()

```

PASO 5: Integrar Sistema de Notificaciones

5.1 Crear notificador adaptado para IRIS:

```

# iris_notifications.py
import smtplib
import requests
import json
from email.mime.text import MIMEText
from email.mime.multipart import MIMEMultipart
from typing import Dict, Any
import logging

class IRISNotificationManager:
    def __init__(self):
        self.smtp_server = "smtp.gmail.com"
        self.smtp_port = 587
        self.email_user = None
        self.email_password = None
        self.webhook_url = None

    def configure_email(self, email: str, password: str):
        """Configurar notificaciones por email"""
        self.email_user = email
        self.email_password = password

    def configure_webhook(self, url: str):
        """Configurar webhook para notificaciones"""
        self.webhook_url = url

    def send_agent_status_notification(self, agent_name: str, status: str, details: Dict[str, Any]):
        """Enviar notificación de cambio de estado de agente"""
        message = {
            "agent": agent_name,
            "status": status,
            "timestamp": details.get('timestamp'),
            "tasks_completed": details.get('tasks_completed', 0),
            "response_time": details.get('avg_response_time', 0)
        }

        # Notificación por email
        if self.email_user:
            self._send_email_notification(f"IRIS Agent Update: {agent_name}", message)

```

```

# Notificación por webhook
if self.webhook_url:
    self._send_webhook_notification(message)

# Notificación por consola
self._send_console_notification(message)

def _send_email_notification(self, subject: str, message: Dict[str,
Any]):
    """Enviar notificación por email"""
    try:
        msg = MIMEMultipart()
        msg['From'] = self.email_user
        msg['To'] = self.email_user
        msg['Subject'] = subject

        body = f"""
        <h2>IRIS Agent Notification</h2>
        <p><strong>Agent:</strong> {message['agent']}</p>
        <p><strong>Status:</strong> {message['status']}</p>
        <p><strong>Tasks Completed:</strong>
{message['tasks_completed']}</p>
        <p><strong>Avg Response Time:</strong>
{message['response_time']}</p>
        <p><strong>Timestamp:</strong> {message['timestamp']}</p>
        """

        msg.attach(MIMEText(body, 'html'))

        server = smtplib.SMTP(self.smtp_server, self.smtp_port)
        server.starttls()
        server.login(self.email_user, self.email_password)
        server.send_message(msg)
        server.quit()

    except Exception as e:
        logging.error(f"Error sending email notification: {e}")

def _send_webhook_notification(self, message: Dict[str, Any]):
    """Enviar notificación por webhook"""
    try:
        response = requests.post(

```



```

        self.webhook_url,
        json=message,
        headers={'Content-Type': 'application/json'})
    )
    response.raise_for_status()
except Exception as e:
    logging.error(f"Error sending webhook notification: {e}")

def _send_console_notification(self, message: Dict[str, Any]):
    """Enviar notificación por consola"""
    status_icon = "🟢" if message['status'] == 'active' else "🔴"
    print(f"{status_icon} IRIS Alert: {message['agent']} status: {message['status']}")

```

PASO 6: Implementar Setup Wizard

6.1 Crear wizard de configuración para IRIS:

```

# iris_setup_wizard.py
import os
import sys
import platform
import subprocess
from pathlib import Path
from typing import Dict, Any, List

class IRISSetupWizard:
    def __init__(self):
        self.config = {
            "api_base": "http://localhost:8000",
            "email_notifications": False,
            "webhook_notifications": False,
            "agents_enabled": [],
            "templates": []
        }

    def detect_system(self) -> Dict[str, Any]:
        """Detectar información del sistema"""
        return {
            "os": platform.system(),
            "python_version": sys.version,
            "node_available": self._check_node(),
            "npm_available": self._check_npm(),
            "git_available": self._check_git()
        }

    def _check_node(self) -> bool:
        """Verificar si Node.js está disponible"""
        try:
            result = subprocess.run(['node', '--version'],
                                    capture_output=True, text=True)
            return result.returncode == 0
        except FileNotFoundError:
            return False

    def _check_npm(self) -> bool:
        """Verificar si npm está disponible"""
        try:
            result = subprocess.run(['npm', '--version'],
                                    capture_output=True, text=True)

```

```

        return result.returncode == 0
    except FileNotFoundError:
        return False

def _check_git(self) -> bool:
    """Verificar si git está disponible"""
    try:
        result = subprocess.run(['git', '--version'],
                                capture_output=True, text=True)
        return result.returncode == 0
    except FileNotFoundError:
        return False

def check_dependencies(self) -> List[str]:
    """Verificar dependencias necesarias"""
    missing = []

    # Verificar Python packages
    required_packages = [
        'fastapi', 'uvicorn', 'requests', 'click'
    ]

    for package in required_packages:
        try:
            __import__(package)
        except ImportError:
            missing.append(f"python-{package}")

    # Verificar Node.js tools
    if not self._check_node():
        missing.append("nodejs")

    if not self._check_npm():
        missing.append("npm")

    return missing

def install_dependencies(self, missing: List[str]) -> bool:
    """Instalar dependencias faltantes"""
    for dep in missing:
        if dep.startswith("python-"):
            package = dep.replace("python-", "")

```

```

        try:
            subprocess.run([sys.executable, "-m", "pip",
"install", package])
            print(f"✅ Instalado: {package}")
        except:
            print(f"❌ Error instalando: {package}")
            return False
    elif dep == "nodejs":
        print("📦 Instala Node.js desde: https://nodejs.org/")
        return False
    elif dep == "npm":
        print("📦 Instala npm con Node.js")
        return False

    return True

def configure_iris(self):
    """Configurar IRIS MCP integration"""
    print("🔧 Configurando integración MCP Server para IRIS")

    # Configurar API base
    api_base = input("🔗 URL de la API de IRIS (default: http://localhost:8000): ").strip()
    if api_base:
        self.config["api_base"] = api_base

    # Configurar notificaciones por email
    email_choice = input("✉️ ¿Configurar notificaciones por email? (y/n): ").strip().lower()
    if email_choice == 'y':
        email = input("✉️ Email: ")
        password = input("🔑 App password: ")
        self.config["email_notifications"] = True

    # Configurar webhook
    webhook_choice = input("🔗 ¿Configurar webhook? (y/n): ").strip().lower()
    if webhook_choice == 'y':
        webhook_url = input("🔗 URL del webhook: ")
        self.config["webhook_notifications"] = True

    # Habilitar agentes

```

```

print("🤖 Agentes disponibles:")
print("1. Sales Agent")
print("2. Support Agent")
print("3. Consulting Agent")

agents_choice = input("Selecciona agentes a habilitar (1-3,
separados por comas): ")
enabled_agents = agents_choice.split(",")

if "1" in enabled_agents:
    self.config["agents_enabled"].append("Sales Agent")
if "2" in enabled_agents:
    self.config["agents_enabled"].append("Support Agent")
if "3" in enabled_agents:
    self.config["agents_enabled"].append("Consulting Agent")

# Guardar configuración
self.save_config()

def save_config(self):
    """Guardar configuración"""
    config_path = Path("iris_mcp_config.json")
    import json

    with open(config_path, 'w') as f:
        json.dump(self.config, f, indent=2)

    print(f"✅ Configuración guardada en: {config_path}")

def run_setup(self):
    """Ejecutar wizard completo"""
    print("🚀 IRIS MCP Server Setup Wizard")
    print("="*50)

    # 1. Detectar sistema
    system_info = self.detect_system()
    print(f"💻 Sistema detectado: {system_info['os']}")
    print(f"🐍 Python: {system_info['python_version']}")

    # 2. Verificar dependencias
    missing = self.check_dependencies()
    if missing:

```

```
        print(f"❌ Dependencias faltantes: {' '.join(missing)}")

        install_choice = input("¿Instalar dependencias faltantes?
(y/n): ")
        if install_choice.lower() == 'y':
            if self.install_dependencies(missing):
                print("✅ Dependencias instaladas correctamente")
            else:
                print("❌ Error instalando dependencias")
                return
        else:
            print("✅ Todas las dependencias están disponibles")

# 3. Configurar IRIS
self.configure_iris()

print("\n🎉 ¡Setup completado exitosamente!")
print("\nPróximos pasos:")
print("1. Ejecutar dashboard: python iris_metrics_server.py")
print("2. Ejecutar CLI: python iris_cli.py status")
print("3. Visitar dashboard: http://localhost:3000")
```

INTEGRACIÓN CON IRIS

API Endpoints a Implementar:

```

# iris_api_integration.py
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from typing import List, Dict, Any
import asyncio

class IRISAPIManager:
    def __init__(self, iris_api_base: str):
        self.iris_api_base = iris_api_base
        self.app = FastAPI(title="IRIS MCP Integration")
        self._setup_routes()

    def _setup_routes(self):
        @self.app.get("/agents")
        async def get_agents():
            """Obtener lista de agentes IRIS"""
            # Integrar con API de IRIS
            agents = await self._fetch_iris_agents()
            return {"agents": agents}

        @self.app.get("/agents/{agent_id}/metrics")
        async def get_agent_metrics(agent_id: str):
            """Obtener métricas de agente específico"""
            metrics = await self._fetch_agent_metrics(agent_id)
            if not metrics:
                raise HTTPException(status_code=404, detail="Agent not found")
            return metrics

        @self.app.post("/agents/{agent_id}/deploy")
        async def deploy_agent(agent_id: str):
            """Desplegar agente en IRIS"""
            result = await self._deploy_iris_agent(agent_id)
            return {"status": "deployed", "agent_id": agent_id}

        @self.app.get("/metrics/stream")
        async def metrics_stream():
            """Stream de métricas en tiempo real"""
            # SSE endpoint para dashboard
            pass

    async def _fetch_iris_agents(self) -> List[Dict[str, Any]]:

```



```

    """Obtener agentes desde API de IRIS"""
    # Aquí iría la lógica de integración con la API de IRIS
    return [
        {
            "id": "sales_agent",
            "name": "Sales Agent",
            "status": "active",
            "capabilities": ["lead_qualification",
"proposal_generation"]
        },
        {
            "id": "support_agent",
            "name": "Support Agent",
            "status": "active",
            "capabilities": ["ticket_classification",
"response_generation"]
        },
        {
            "id": "consulting_agent",
            "name": "Consulting Agent",
            "status": "active",
            "capabilities": ["data_analysis", "report_generation"]
        }
    ]

    async def _fetch_agent_metrics(self, agent_id: str) -> Dict[str,
Any]:
        """Obtener métricas de agente específico"""
        # Lógica para obtener métricas de IRIS
        return {
            "agent_id": agent_id,
            "tasks_completed": 156,
            "avg_response_time": 1.2,
            "token_usage": 45200,
            "success_rate": 0.94
        }

```

CONFIGURACIÓN DE APIS

Variables de Entorno:

```
# .env
IRIS_API_BASE=http://your-iris-api.com
IRIS_API_KEY=your_api_key_here
IRIS_SMTP_SERVER=smtp.gmail.com
IRIS_SMTP_PORT=587
IRIS_EMAIL_USER=your_email@gmail.com
IRIS_EMAIL_PASSWORD=your_app_password
IRIS_WEBHOOK_URL=https://your-webhook.com/notifications
IRIS_DASHBOARD_PORT=3000
IRIS_METRICS_PORT=8000
```

Configuración de MiniMax:

```
# minimax_config.py
MINIMAX_CONFIG = {
    "api_base": "https://api.minimax.chat",
    "models": {
        "m1": "minimax/minimax-m1:free",
        "m2": "minimax/minimax-m2:free"
    },
    "capabilities": [
        "text_generation",
        "image_analysis",
        "audio_processing",
        "video_generation"
    ]
}
```

TESTING Y VALIDACIÓN

Test Suite:

```
# test_iris_mcp_integration.py
import unittest
import asyncio
from iris_api_integration import IRISAPIManager

class TestIRISMCPIntegration(unittest.TestCase):
    def setUp(self):
        self.api_manager = IRISAPIManager("http://test-iris-api.com")

    def test_agents_fetch(self):
        """Test obtener agentes"""
        agents = asyncio.run(self.api_manager._fetch_iris_agents())
        self.assertIsInstance(agents, list)
        self.assertGreater(len(agents), 0)

    def test_metrics_fetch(self):
        """Test obtener métricas"""
        metrics =
        asyncio.run(self.api_manager._fetch_agent_metrics("sales_agent"))
        self.assertIn("agent_id", metrics)
        self.assertIn("tasks_completed", metrics)

    def test_agent_deployment(self):
        """Test despliegue de agente"""
        result =
        asyncio.run(self.api_manager._deploy_iris_agent("test_agent"))
        self.assertEqual(result["status"], "deployed")

if __name__ == '__main__':
    unittest.main()
```

Commands de Testing:

```
# Ejecutar tests
python -m pytest test_iris_mcp_integration.py -v

# Test manual de CLI
python iris_cli.py status
python iris_cli.py metrics
python iris_cli.py template sales

# Test de dashboard
cd dashboard
npm run dev
curl http://localhost:8000/agents
```

Docker Setup:

```
# Dockerfile
FROM node:18-alpine
WORKDIR /app

# Instalar dependencias
COPY package*.json ./
RUN npm install

# Copiar código
COPY . .

# Build
RUN npm run build

# Backend
FROM python:3.9-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["uvicorn", "iris_api_integration:app", "--host", "0.0.0.0", "--port", "8000"]
```

Deploy Script:

```
#!/bin/bash
# deploy.sh

echo "🚀 Desplegando IRIS MCP Integration"

# 1. Build dashboard
cd dashboard
npm run build
cd ..

# 2. Install Python dependencies
pip install -r requirements.txt

# 3. Start services
echo "📊 Iniciando servidor de métricas..."
uvicorn iris_api_integration:app --host 0.0.0.0 --port 8000 &

echo "🌐 Iniciando dashboard..."
cd dashboard && npm run dev &

echo "✅ Despliegue completado"
echo "📊 Dashboard: http://localhost:3000"
echo "🔊 API: http://localhost:8000"
```



CHECKLIST DE IMPLEMENTACIÓN

- ☐ Setup del entorno de desarrollo
 - ☐ Implementar Dashboard React con métricas de IRIS
 - ☐ Crear sistema de templates para IRIS
 - ☐ Desarrollar CLI avanzada con comandos específicos
 - ☐ Integrar sistema de notificaciones
 - ☐ Crear Setup Wizard para configuración
 - ☐ Implementar API integration con IRIS
 - ☐ Configurar variables de entorno
 - ☐ Ejecutar tests de integración
 - ☐ Desplegar en producción
-

BENEFICIOS ESPERADOS

Para Usuarios de IRIS:

-  **Visibilidad en tiempo real** del estado de agentes
-  **Automatización avanzada** con templates predefinidos
-  **Gestión centralizada** vía CLI
-  **Notificaciones inteligentes** sobre estado de tareas
-  **Configuración rápida** con setup wizard

Para Desarrolladores:

-  **APIs estándar** para integración
 -  **Documentación completa** de componentes
 -  **Test suite** para validación
 -  **Scripts de despliegue** automatizados
 -  **Configuración flexible** via variables de entorno
-

PRÓXIMOS PASOS

1. **Implementar los componentes paso a paso**
 2. **Testear cada componente individualmente**
 3. **Integrar con API de IRIS**
 4. **Configurar dashboard y métricas**
 5. **Desplegar en entorno de producción**
-

Esta guía te permitirá replicar completamente el MCP Server Superior adaptado específicamente para mejorar la experiencia de usuario de IRIS.